

**Implementasi Shamir Secret Sharing pada Aplikasi Password Manager
Terdistribusi**

LAPORAN TUGAS 4

**Diajukan untuk memenuhi Tugas 4 II4021 Kriptografi
Semester II tahun 2025/2026**

**Oleh Kelompok 24
Jason Samuel (18223091)**



**PROGRAM STUDI SISTEM DAN TEKNOLOGI INFORMASI
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
2026**

DAFTAR ISI

1. Pernyataan Tidak Melakukan Kecurangan	3
2. Teori Singkat	4
i. Shamir's Secret Sharing (2,3)	4
ii. Authenticated Encryption (AES-GCM)	4
iii. Key Derivation Function (KDF)	4
iv. PBKDF2 (Password-Based Key Derivation Function 2)	4
v. CSPRNG (Cryptographically Secure Pseudo-Random Number Generator)	4
3. Perancangan dan Implementasi	5
i. Inisialisasi Medan Kriptografi dan Konstanta Global	5
Code 1. variabel PRIME di file client.py	5
ii. Shamir's Secret Sharing (2,3)	5
Code 2. fungsi split_secret di file client.py	5
Code 3. Fungsi reconstruct_secret di file client.py	5
iii. Enkripsi	6
Code 4. Fungsi derive_key di file client.py	6
Code 5. Fungsi encrypt di file client.py	6
Code 6. Fungsi decrypt di file client.py	6
iv. Local Storage	7
Code 7. Fungsi save_local di file client.py	7
Code 8. Fungsi load_local di file client.py	7
v. Password Generator	8
Code 9. Fungsi getSharedKey di file client/src/App.jsx	8
vi. Server	8
Code 10. Fungsi init_db di file server.py	8
Code 11. Fungsi save_vault di file server.py	9
Code 12. get_vault di file server.py	9
vii. Pembuatan Vault	10
Code 13. Fungsi create_vault di main.py	11
viii. Melihat Isi Vault	11
Code 14. Fungsi view_passwords di main.py	11
ix. Mengubah Isi Vault	11
Code 15. fungsi add_password di main.py	12
Code 16. fungsi update_password di main.py	13
Code 17. fungsi delete_password di main.py	14
x. Toggle Server	14
Code 18. fungsi toggle_server di main.py	14
xi. Normal Mode	14
Code 19. fungsi normal_mode di main.py	15
xii. Backup Mode	16
Code 20. fungsi backup_mode di main.py	16
4. Pengujian Program	17
Code 21. fungsi create_vault di main.py untuk pengujian	18
Code 22. fungsi normal_mode di main.py untuk pengujian	19
Code 23. perubahan fungsi add_password, update_password, delete_password di main.py untuk pengujian	19
Code 24. perubahan fungsi backup_mode di main.py untuk pengujian	20
Gambar 1. bukti uji pembuatan vault 1	20
Gambar 2. isi server_vault.db setelah pembuatan vault	21

Gambar 3. isi client_local_123 setelah pembuatan vault	21
Gambar 4. dekripsi local share setelah input master password yang benar	21
Gambar 5. dekripsi local share setelah input master password yang salah	22
Gambar 6. testing normal_mode	22
Gambar 7. testing normal_mode dengan vault yang terisi	22
Gambar 8. testing normal_mode dengan master password yang salah	22
Gambar 9. penambahan password manual ke vault	23
Gambar 10. penambahan password auto generated ke vault	23
Gambar 11. Isi local vault setelah penambahan password pada gambar 9&10	24
Gambar 12. Isi server vault setelah penambahan password pada gambar 9&10	24
Gambar 13. perubahan password	24
Gambar 14. penghapusan password	25
Gambar 15. Testing Backup Mode	25
Gambar 16. Toggle Server Off dan penggunaan Backup Mode di kondisi server off	26
Gambar 17. Penggunaan Backup Mode di dengan recovery share yang salah	26
Gambar 18. Toggle Server Off dan penggunaan Backup Mode dengan perubahan isi backup vault	27
1. Uji pembuatan vault	27
2. Uji penyimpanan dan perlindungan local share	28
3. Uji Akses Normal	29
4. Uji Penambahan Data Password	29
5. Uji Pengubahan dan Penghapusan Data Password	30
6. Uji Penyimpanan Vault di Server	30
7. Uji Mode Backup	31
8. Uji Kegagalan Pemulihan	31
5. Kesimpulan dari Hasil Implementasi	32
6. Daftar Pustaka	33
1. A. Shamir, "How to share a secret," Communications of the ACM, vol. 22, no. 11, pp. 612–613, Nov. 1979.	33
2. D. A. McGrew and J. Viega, "The Galois/Counter Mode of Operation (GCM)," Submission to NIST Modes of Operation Process, pp. 1–44, May 2004.	33
3. M. Dworkin, "Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC," National Institute of Standards and Technology (NIST), Gaithersburg, MD, USA, NIST Special Publication 800-38D, Nov. 2007.	33
4. B. Kaliski, "PKCS #5: Password-Based Cryptography Specification Version 2.0," RFC 2898, Sep. 2000.	33
5. K. Moriarty, B. Kaliski, and A. Rusch, "PKCS #5: Password-Based Cryptography Specification Version 2.1," RFC 8018, Internet Engineering Task Force (IETF), 2017.	33
6. E. Barker and J. Kelsey, "Recommendation for Random Number Generation Using Deterministic Random Bit Generators," National Institute of Standards and Technology (NIST), Gaithersburg, MD, USA, NIST Special Publication 800-90A Rev. 1, Jun. 2015.	33
7. Lampiran	34
1. Pranala Repositori	34
2. Pranala Video Demo	34
3. Pembagian Tugas	34

1. Pernyataan Tidak Melakukan Kecurangan

Kami menyatakan bahwa kode program yang dihasilkan bukan merupakan hasil salinan mentah (raw output) dari Generative AI, melainkan hasil pengembangan dan penulisan mandiri.



2. Teori Singkat

i. Shamir's Secret Sharing (2,3)

Skema kriptografi ambang batas (threshold cryptography) yang memecah sebuah data rahasia (master key) menjadi N buah bagian (shares). Melalui pendekatan interpolasi polinomial, rahasia asli hanya dapat dikonstruksi kembali jika minimal terdapat k bagian pecahan yang digabungkan (dalam program ini, minimal 2 dari 3 bagian ($N=3$, $k=2$)), sehingga kehilangan satu pecahan tidak akan membocorkan informasi rahasia apa pun.

ii. Authenticated Encryption (AES-GCM)

Metode enkripsi simetris yang menggabungkan algoritma Advanced Encryption Standard (AES) dengan Galois/Counter Mode (GCM). Protokol ini menyediakan layanan keamanan ganda dalam satu proses tunggal, yaitu menjaga kerahasiaan data brankas (konfidensialitas) sekaligus menjamin bahwa data tersebut tidak mengalami modifikasi ilegal oleh pihak ketiga (integritas).

iii. Key Derivation Function (KDF)

Sebuah fungsi penurunan kunci berbasis kata sandi yang digunakan untuk memperkuat keamanan kata sandi utama (master password) pengguna. Dengan menerapkan fungsi hashing kriptografi secara berulang (>10.000 iterasi) dikombinasikan dengan nilai acak unik (salt), metode ini secara efektif meningkatkan resistensi kunci terhadap serangan brute-force dan rainbow table.

iv. PBKDF2 (Password-Based Key Derivation Function 2)

Protokol spesifik dari KDF yang dirancang khusus untuk memperkuat keamanan kata sandi utama (master password) pengguna. Dengan menerapkan fungsi hashing kriptografi secara berulang (100.000 iterasi pada program) dikombinasikan dengan nilai acak unik (salt), metode ini secara efektif memperlambat komputasi guna meningkatkan resistensi kunci terhadap serangan brute-force.

v. CSPRNG (Cryptographically Secure Pseudo-Random Number Generator)

Sebuah generator angka acak semu yang memenuhi standar keamanan kriptografi, yang pada program ini diimplementasikan melalui fungsi `get_random_bytes`. CSPRNG menjamin bahwa nilai acak yang dihasilkan (seperti salt, nonce, dan master key) memiliki sifat tidak dapat diprediksi (unpredictability) dan tidak memiliki pola yang dapat dieksploitasi oleh penyerang.

3. Perancangan dan Implementasi

i. Inisialisasi Medan Kriptografi dan Konstanta Global

```
PRIME = 2**128 + 51
```

Code 1. variabel PRIME di file client.py

Pada baris awal di file client.py, program menetapkan sebuah konstanta bilangan prima besar. Nilai ini berfungsi sebagai basis aritmatika modular (Medan Galois) yang membungkus seluruh perhitungan numerik dalam skema pembagian rahasia. Penetapan bilangan prima ini sangat penting agar hasil operasi matematika seperti pembagian dan perkalian tetap menghasilkan nilai bulat unik yang dapat dibalikkan (reversible) saat proses rekonstruksi kunci, serta mencegah kebocoran informasi akibat pola angka yang terprediksi.

ii. Shamir's Secret Sharing (2,3)

```
def split_secret(secret_bytes):
    secret = int.from_bytes(secret_bytes, "big")
    a = int.from_bytes(get_random_bytes(16), "big") % PRIME

    shares = []
    for x in [1, 2, 3]:
        y = (a * x + secret) % PRIME
        shares.append({
            "x": x,
            "y": hex(y)
        })

    return shares
```

Code 2. fungsi split_secret di file client.py

Fungsi split_secret bertugas mengimplementasikan algoritma interpolasi polinomial linear untuk memecah kunci utama (master key). Pertama-tiba, kunci utama berbentuk biner diubah menjadi representasi bilangan bulat (integer) besar. Selanjutnya, program memanfaatkan generator angka acak aman (CSPRNG) untuk membangkitkan nilai gradien atau koefisien acak (\$a\$). Dengan menggunakan persamaan garis lurus $y = ax + \text{secret}$ dalam modulo bilangan prima, program menghitung tiga koordinat titik (x, y) yang berbeda dengan nilai x berupa 1, 2, dan 3. Ketiga koordinat inilah yang kemudian didistribusikan sebagai pecahan komponen lokal, komponen server, dan komponen pemulihan.

```
def reconstruct_secret(s1, s2):
    x1, y1 = s1["x"], int(s1["y"], 16)
    x2, y2 = s2["x"], int(s2["y"], 16)

    denom = (x2 - x1) % PRIME
    inv = pow(denom, PRIME - 2, PRIME)

    secret = (y1 * x2 * inv - y2 * x1 * inv) % PRIME
    return secret.to_bytes(16, "big")
```

Code 3. Fungsi reconstruct_secret di file client.py

Fungsi reconstruct_secret merupakan komplemen dari fungsi pembagian, yang bertujuan menyatukan kembali dua buah pecahan koordinat titik untuk menemukan intersep-y (nilai saat $x = 0$), yang tidak lain adalah kunci utama asli. Di dalam fungsinya, program melakukan ekstraksi data koordinat dan menghitung nilai kemiringan atau penyebut dari kedua titik tersebut. Melalui Teorema Kecil Fermat, program menghitung invers perkalian modular dari penyebut tersebut agar operasi pembagian dalam medan bilangan prima dapat dilakukan dengan valid. Setelah perhitungan intersep selesai, nilai integer yang didapatkan dikonversi kembali menjadi deretan data biner (bytes) sepanjang 16 byte.

iii. Enkripsi

```
def derive_key(password, salt):  
    return PBKDF2(password, salt, dkLen=16, count=100000)
```

Code 4. Fungsi derive_key di file client.py

Fungsi derive_key bertindak sebagai mekanisme penguat kata sandi menggunakan fungsi penurunan kunci berbasis password (PBKDF2). Fungsi ini menerima masukan berupa kata sandi teks biasa dari pengguna beserta nilai acak unik (salt). Di dalamnya, program menjalankan fungsi hashing kriptografi berulang-ulang sebanyak 100.000 iterasi untuk memperlambat waktu komputasi secara sengaja. Hasil akhir dari fungsi ini adalah kunci simetris sepanjang 16 byte yang memiliki tingkat acak tinggi, menjadikannya sangat tangguh terhadap serangan tebakan cepat seperti brute-force atau pencocokan tabel hashing (rainbow tables).

```
def encrypt(plaintext, key):  
    cipher = AES.new(key, AES.MODE_GCM)  
    ct, tag = cipher.encrypt_and_digest(plaintext)  
    return ct + tag, cipher.nonce
```

Code 5. Fungsi encrypt di file client.py

Fungsi encrypt bertanggung jawab untuk mengamankan data (baik pecahan kunci lokal maupun isi list brankas kata sandi) menggunakan algoritma AES dengan mode operasi GCM (Galois/Counter Mode). Saat dijalankan, fungsi ini secara otomatis membangkitkan nilai nonce (angka acak unik yang hanya boleh digunakan sekali) untuk memastikan hasil enkripsi selalu unik. Program kemudian mengenkripsi data sekaligus menghasilkan sebuah tag autentikasi sepanjang 16 byte. Fungsi ini mengembalikan sebuah paket data biner gabungan antara teks terenkripsi (cipherteks) dengan tag autentikasi tersebut, bersamaan dengan nilai nonce-nya.

```
def decrypt(blob, key, nonce):  
    cipher = AES.new(key, AES.MODE_GCM, nonce=nonce)  
    ct, tag = blob[:-16], blob[-16:]  
    return cipher.decrypt_and_verify(ct, tag)
```

Code 6. Fungsi decrypt di file client.py

Fungsi decrypt merupakan kebalikan dari proses enkripsi yang bertugas membuka kembali data yang terkunci. Fungsi ini menerima paket data terenkripsi, kunci dekripsi, dan nilai nonce yang sesuai. Di dalam operasinya, program memotong 16 byte terakhir dari paket data untuk memisahkan komponen tag autentikasi dari cipherteks-nya. Menggunakan parameter tersebut, objek AES-GCM mencoba mendekripsi data sekaligus memverifikasi keasliannya. Jika data brankas terbukti asli dan belum pernah dimodifikasi sama sekali, data teks biasa (plaintext) akan dikembalikan; namun jika verifikasi tag gagal, fungsi akan melemparkan kesalahan (exception) dan membatalkan proses.

iv. Local Storage

```
def save_local(username, enc_share, nonce, salt, backup_vault,
backup_nonce):
    filename = f"client_local_{username}.json"

    data = {
        "enc_share": base64.b64encode(enc_share).decode(),
        "nonce": base64.b64encode(nonce).decode(),
        "salt": base64.b64encode(salt).decode(),
        "backup_vault": base64.b64encode(backup_vault).decode(),
        "backup_nonce": base64.b64encode(backup_nonce).decode()
    }

    with open(filename, "w") as f:
        json.dump(data, f)
```

Code 7. Fungsi save_local di file client.py

fungsi save_local menangani manajemen berkas pada sisi klien untuk menyimpan status data secara persisten ke dalam media penyimpanan fisik komputer. Karena format file JSON tidak dapat menampung data biner mentah secara langsung, fungsi save_local mengubah seluruh variabel biner (seperti data terenkripsi, nonce, dan salt) menjadi representasi teks string berbasis Base64 sebelum ditulis ke dalam file berformat client_local_[username].json.

```
def load_local(username):
    filename = f"client_local_{username}.json"

    if not os.path.exists(filename):
        return None

    with open(filename, "r") as f:
        data = json.load(f)

    return {
        "enc_share": base64.b64decode(data["enc_share"]),
        "nonce": base64.b64decode(data["nonce"]),
        "salt": base64.b64decode(data["salt"]),
        "backup_vault": base64.b64decode(data["backup_vault"]),
        "backup_nonce": base64.b64decode(data["backup_nonce"])
    }
```

Code 8. Fungsi load_local di file client.py

Sebaliknya, fungsi load_local bertugas memeriksa keberadaan file tersebut, membacanya, dan menerjemahkan kembali teks string Base64 tersebut ke dalam bentuk data biner asli agar siap diproses oleh fungsi-fungsi kriptografi.

v. Password Generator

```
def generate_password(length):
    chars = string.ascii_letters + string.digits + "!@#$$%^&*()"
    return "".join(
        chars[b % len(chars)]
```

```

        for b in get_random_bytes(length)
    )

```

Code 9. Fungsi getSharedKey di file client/src/App.jsx

Fungsi generate_password menyediakan utilitas praktis bagi pengguna untuk menciptakan kata sandi baru yang acak dan kuat. Program mendefinisikan kumpulan karakter standar yang terdiri dari huruf besar, huruf kecil, angka, dan simbol khusus. Fungsi ini kemudian mengambil sejumlah byte acak dari generator angka acak aman (CSPRNG) sesuai dengan panjang kata sandi yang diinginkan. Setiap byte acak tersebut dipetakan ke dalam indeks kumpulan karakter menggunakan operasi matematika modulo (%), lalu digabungkan menjadi satu kesatuan string kata sandi baru.

vi. Server

```

DB_FILE = "server_vault.db"
SERVER_ACTIVE = True

def init_db():
    conn = sqlite3.connect(DB_FILE)
    c = conn.cursor()
    c.execute("""
CREATE TABLE IF NOT EXISTS user_vault (
    username TEXT PRIMARY KEY,
    server_share TEXT NOT NULL,
    vault_blob BLOB NOT NULL,
    nonce BLOB NOT NULL
)
""")
    conn.commit()
    conn.close()

```

Code 10. Fungsi init_db di file server.py

Fungsi init_db bertanggung jawab penuh untuk meletakkan fondasi struktur data pada saat aplikasi pertama kali dijalankan oleh pengguna. Di dalam fungsinya, program membuka jalur koneksi ke file database melalui pustaka SQLite, yang secara cerdas akan otomatis membuat file baru jika file tersebut belum eksis di komputer. Menggunakan objek cursor, fungsi ini mengeksekusi perintah SQL CREATE TABLE IF NOT EXISTS untuk membangun sebuah tabel relasional bernama user_vault. Tabel ini dirancang dengan cetak biru empat kolom spesifik: username sebagai kunci utama yang unik, server_share untuk menampung teks string pecahan rahasia, serta vault_blob dan nonce untuk menyimpan objek data biner terenkripsi. Setelah eksekusi perintah selesai, perubahan struktur tersebut disimpan secara permanen ke piringan cakram keras komputer melalui perintah commit sebelum koneksi akhirnya ditutup dengan aman.

```

def save_vault(username, server_share, vault_blob, nonce):
    if not SERVER_ACTIVE:
        return None
    conn = sqlite3.connect(DB_FILE)
    c = conn.cursor()
    c.execute("""

```

```

        INSERT OR REPLACE INTO user_vault (username, server_share,
        vault_blob, nonce)
        VALUES (?, ?, ?, ?)
        """ , (username, server_share, vault_blob, nonce))
        conn.commit()
        conn.close()

```

Code 11. Fungsi save_vault di file server.py

Fungsi save_vault memegang peran krusial dalam memfasilitasi penulisan data baru hasil registrasi maupun sinkronisasi pembaruan isi brankas ketika pengguna melakukan manipulasi kata sandi. Sebelum melakukan operasi baca-tulis ke database, fungsi ini melakukan pemeriksaan wajib terhadap status variabel SERVER_ACTIVE. Jika dideteksi bahwa server sedang dalam kondisi mati (bernilai False), fungsi ini akan langsung memotong jalur eksekusi dan mengembalikan nilai kosong (None) sehingga data gagal disimpan ke database. Namun jika server aktif, fungsi akan membuka koneksi dan mengeksekusi kueri SQL INSERT OR REPLACE. Klausula ini memastikan bahwa jika nama pengguna belum terdaftar maka baris data baru akan ditambahkan, sedangkan jika nama pengguna sudah ada maka seluruh data biner dan pecahan kunci lama miliknya akan langsung ditimpa dengan data terbaru yang dikirim oleh klien. Guna menangkal celah keamanan dari serangan siber, fungsi ini menerapkan teknik Prepared Statements menggunakan parameter tanda tanya untuk memisahkan logika kueri dari input mentah pengguna.

```

def get_vault(username):
    if not SERVER_ACTIVE:
        return None
    conn = sqlite3.connect(DB_FILE)
    c = conn.cursor()
    c.execute("""
    SELECT server_share, vault_blob, nonce
    FROM user_vault
    WHERE username = ?
    """, (username,))
    row = c.fetchone()
    conn.close()
    if not row:
        return None

    return {
        "server_share": row[0],
        "vault_blob": row[1],
        "nonce": row[2]
    }

```

Code 12. get_vault di file server.py

Fungsi get_vault bertindak sebagai penyedia layanan penarikan informasi rahasia terenkripsi dari server ketika pengguna mencoba melakukan proses masuk ke dalam aplikasi. Sama seperti fungsi penyimpanan, fungsi ini mengawali kerjanya dengan melakukan validasi status keaktifan server untuk memastikan simulasi jaringan berjalan konsisten. Apabila server sedang luring, fungsi ini mengembalikan nilai None secara instan, yang mana nilai kosong ini akan ditangkap oleh fungsi utama aplikasi untuk mendeteksi matinya server dan langsung mengalihkan pengguna ke mode darurat (Backup Mode). Jika server dalam kondisi normal,

fungsi akan menjalankan kueri SQL SELECT untuk mencari data spesifik berdasarkan parameter nama pengguna yang diminta. Hasil pencarian baris data tersebut kemudian diekstraksi dan dikemas ke dalam sebuah objek Dictionary Python yang rapi—memetakan data koordinat pecahan server, gumpalan brankas terenkripsi, beserta nonce pasangannya—sebelum diserahkan kembali kepada klien untuk diproses ke tahap rekonstruksi kunci.

vii. Pembuatan Vault

```
def create_vault():
    username = input("Username: ").strip()
    password = input("Master Password: ").strip()

    if not username or not password:
        print("Invalid input")
        return

    if server.get_vault(username):
        print("Username already exists. Use login instead.")
        return

    if not server.SERVER_ACTIVE:
        print("Server offline. Cannot create vault.")
        return

    master_key = get_random_bytes(16)

    local_share, server_share, recovery_share =
client.split_secret(master_key)

    salt = get_random_bytes(16)
    kdf_key = client.derive_key(password, salt)

    enc_local, nonce_local = client.encrypt(
        json.dumps(local_share).encode(),
        kdf_key
    )

    vault = []

    enc_vault, nonce_vault = client.encrypt(
        json.dumps(vault).encode(),
        master_key
    )

    server.save_vault(
        username,
        json.dumps(server_share),
        enc_vault,
        nonce_vault
    )

    client.save_local(
        username,
        enc_local,
        nonce_local,
        salt,
        enc_vault,
        nonce_vault
    )

    print("\nVAULT CREATED SUCCESSFULLY")
    print("RECOVERY SHARE (SAVE THIS ONCE):")
    print(json.dumps(recovery_share))
```

Code 13. Fungsi create_vault di main.py

Fungsi create_vault mengendalikan alur logika registrasi awal pengguna baru di dalam sistem. Setelah memvalidasi input dan memastikan server sedang aktif, fungsi ini menghasilkan sebuah kunci utama (master key) acak dari CSPRNG dan memecahnya menjadi tiga bagian rahasia. Program kemudian membuat salt baru, menurunkan kunci dari password utama lewat KDF, dan mengenkripsi pecahan kunci lokal. Fungsi ini juga membuat struktur brankas kosong, mengenkripsinya dengan kunci utama, lalu membagi tugas penyimpanan: pecahan server beserta data brankas terenkripsi dikirim ke database server, sedangkan pecahan lokal terenkripsi beserta salinan brankas terenkripsi disimpan di PC klien. Di akhir proses, fungsi ini menampilkan kode pecahan pemulihan (recovery share) di layar agar dicatat oleh pengguna.

viii. Melihat Isi Vault

```
view_passwords(vault):
    print("\n--- YOUR VAULT ---")
    for i, item in enumerate(vault):
        cat = item.get("catatan", "")
        extra = f" | Catatan: {cat}" if cat else ""
        print(f"[{i}] {item['nama_layanan']} | {item['username']} | {item['password']}{extra}")
```

Code 14. Fungsi view_passwords di main.py

Fungsi view_passwords memiliki kegunaan sederhana namun krusial, yaitu menampilkan seluruh isi brankas kata sandi yang telah berhasil didekripsi ke layar pengguna. Fungsi ini melakukan iterasi atau perulangan pada list objek brankas, membaca atribut seperti nama layanan, username, kata sandi, dan catatan tambahan, lalu mencetaknya dalam format baris teks yang rapi dan terstruktur lengkap dengan nomor indeksinya. Nomor indeks ini nantinya digunakan oleh pengguna sebagai acuan jika ingin mengubah atau menghapus item tertentu.

ix. Mengubah Isi Vault

```
def add_password(vault, key, username, server_share, local):
    layanan = input("Service: ").strip()
    user = input("Username: ").strip()

    print("1. Manual Password")
    print("2. Auto Generate")
    opt = input("Choice: ")

    if opt == "1":
        pw = input("Password: ").strip()
    else:
        length = int(input("Length: "))
        pw = client.generate_password(length)

    catatan = input("Catatan: ").strip()

    vault.append({
        "nama_layanan": layanan,
        "username": user,
        "password": pw,
        "catatan": catatan
    })
```

```

        "catatan": catatan
    })

    enc, nonce = client.encrypt(json.dumps(vault).encode(), key)

    server.save_vault(username, json.dumps(server_share), enc, nonce)

    client.save_local(
        username,
        local["enc_share"],
        local["nonce"],
        local["salt"],
        enc,
        nonce
    )

    print("ADDED")

```

Code 15. fungsi add_password di main.py

Fungsi add_password bertanggung jawab untuk memfasilitasi pengguna dalam memasukkan data akun atau kredensial baru ke dalam sistem manajemen kata sandi. Di awal proses, fungsi ini meminta input dari pengguna berupa nama layanan (seperti nama situs web atau aplikasi) dan nama pengguna (username) terkait. Selanjutnya, fungsi memberikan dua opsi pembuatan kata sandi: pengguna dapat mengetikkan kata sandinya secara manual atau memanfaatkan fitur pembuat kata sandi otomatis (generate_password) yang akan menghasilkan kombinasi karakter acak yang kuat menggunakan generator angka acak aman (CSPRNG). Pengguna juga diberikan opsi untuk menambahkan deskripsi singkat pada kolom catatan. Setelah seluruh data terkumpul, fungsi akan memasukkannya ke dalam struktur data list brankas (vault) di dalam memori, lalu segera mengenkripsi ulang list tersebut menggunakan kunci utama (master key) aktif melalui algoritma AES-GCM. Untuk menjaga konsistensi data, hasil enkripsi terbaru ini langsung dikirim ke database server melalui fungsi save_vault dan secara simultan memperbarui file cadangan JSON lokal milik klien.

```

def update_password(vault, key, username, server_share, local):
    view_passwords(vault)
    idx = input("Index (empty to cancel): ").strip()

    if idx == "":
        print("Cancelled")
        return

    if not idx.isdigit():
        print("Invalid input. Must be a number")
        return

    i = int(idx)

    if i < 0 or i >= len(vault):
        print("Invalid index")
        return

    new_user = input("New username: ").strip()
    new_pw = input("New password: ").strip()
    new_cat = input("New catatan: ").strip()

    if new_user:
        vault[i]["username"] = new_user
    if new_pw:
        vault[i]["password"] = new_pw
    if new_cat:

```

```

        vault[i]["catatan"] = new_cat

    enc, nonce = client.encrypt(json.dumps(vault).encode(), key)

    server.save_vault(username, json.dumps(server_share), enc, nonce)

    client.save_local(
        username,
        local["enc_share"],
        local["nonce"],
        local["salt"],
        enc,
        nonce
    )

    print("UPDATED")

```

Code 16. fungsi update_password di main.py

Fungsi update_password dirancang untuk memberikan kemampuan kepada pengguna dalam mengubah data kredensial yang sudah tersimpan sebelumnya di dalam brankas. Ketika fungsi ini dipanggil, program akan terlebih dahulu menjalankan fungsi penampil (view_passwords) untuk memunculkan seluruh daftar akun terenkripsi yang berhasil dibuka, lengkap dengan nomor indeksinya masing-masing. Pengguna kemudian diminta memasukkan nomor indeks dari akun yang ingin diubah, di mana fungsi akan melakukan validasi ketat untuk memastikan bahwa input tersebut berupa angka dan berada dalam rentang indeks list yang valid. Jika validasi lolos, pengguna diminta memasukkan data baru untuk nama pengguna, kata sandi, dan catatan. Fungsi ini menerapkan sistem pembaruan opsional; jika pengguna mengosongkan input pada salah satu kolom, data lama pada kolom tersebut tidak akan diubah. Setelah modifikasi pada objek list brankas selesai dilakukan di memori, fungsi akan mengunci kembali seluruh brankas menggunakan kunci utama (master key) dengan algoritma AES-GCM, lalu mengirimkan data terbaru tersebut ke database server serta memperbarui file JSON lokal klien.

```

def delete_password(vault, key, username, server_share, local):
    view_passwords(vault)
    idx = input("Index (empty to cancel): ").strip()

    if idx == "":
        print("Cancelled")
        return

    if not idx.isdigit():
        print("Invalid input. Must be a number")
        return

    i = int(idx)

    if i < 0 or i >= len(vault):
        print("Invalid index")
        return
    vault.pop(i)

    enc, nonce = client.encrypt(json.dumps(vault).encode(), key)

    server.save_vault(username, json.dumps(server_share), enc, nonce)

    client.save_local(
        username,

```

```

        local["enc_share"],
        local["nonce"],
        local["salt"],
        enc,
        nonce
    )

    print("DELETED")

```

Code 17. fungsi delete_password di main.py

Fungsi delete_password bertindak sebagai mekanisme untuk mengeliminasi atau menghapus secara permanen data akun yang tidak lagi dibutuhkan oleh pengguna dari dalam brankas. Alur kerja fungsi ini diawali dengan menampilkan seluruh isi brankas kata sandi saat ini ke layar pengguna menggunakan fungsi penampil untuk memberikan referensi visual yang jelas mengenai item yang tersedia. Pengguna kemudian diminta untuk menginput nomor indeks spesifik dari item yang ingin dihapus. Fungsi akan memeriksa input tersebut; jika pengguna menekan tombol enter tanpa memasukkan angka, proses penghapusan otomatis dibatalkan demi keamanan. Program juga memastikan bahwa nomor indeks yang dimasukkan benar-benar ada di dalam list. Begitu validasi terpenuhi, fungsi akan menghapus item tersebut dari list brankas di dalam memori menggunakan metode .pop(). Sama seperti proses penambahan dan pembaruan, list brankas yang kini sudah berkurang satu item akan langsung dienkrpsi ulang menggunakan kunci utama (master key), lalu gumpalan data terenkrpsi (blob) dan nonce barunya dikirim ke database server serta ditulis ke file penyimpanan JSON lokal klien agar perubahan bersifat permanen.

x. Toggle Server

```

def toggle_server():
    server.SERVER_ACTIVE = not server.SERVER_ACTIVE
    print("SERVER ACTIVE:", server.SERVER_ACTIVE)

```

Code 18. fungsi toggle_server di main.py

Fungsi toggle_server berfungsi mengubah status variabel boolean keaktifan server. Fungsi ini digunakan untuk mensimulasikan kondisi jaringan putus atau server luring (offline).

xi. Normal Mode

```

def normal_mode():
    if not server.SERVER_ACTIVE:
        print("Server offline → switching to backup mode")
        backup_mode()

    username = input("Username: ")
    password = input("Master Password: ")

    srv = server.get_vault(username)
    if not srv:
        print("User not found")
        return

```

```

        local = client.load_local(username)
        if not local:
            print("Local data missing")
            return

        try:
            kdf_key = client.derive_key(password, local["salt"])

            local_share = json.loads(
                client.decrypt(local["enc_share"], kdf_key,
local["nonce"])
            )
        except:
            print("AUTH FAILED")
            return

        try:
            server_share = json.loads(srv["server_share"])

            master_key = client.reconstruct_secret(local_share,
server_share)

            vault = json.loads(
                client.decrypt(srv["vault_blob"], master_key,
srv["nonce"])
            )
        except Exception as e:
            print("DECRYPT ERROR:", e)
            return

        while True:
            print("\n1. View")
            print("2. Add")
            print("3. Update")
            print("4. Delete")
            print("5. Exit")

            ch = input("> ")

            if ch == "1":
                view_passwords(vault)
            elif ch == "2":
                add_password(vault, master_key, username, server_share,
local)
            elif ch == "3":
                update_password(vault, master_key, username,
server_share, local)
            elif ch == "4":
                delete_password(vault, master_key, username,
server_share, local)
            elif ch == "5":
                break

```

Code 19. fungsi normal_mode di main.py

Fungsi normal_mode() bertanggung jawab penuh untuk mengendalikan alur masuk (login) utama pengguna ke dalam aplikasi ketika sistem berjalan terhubung dengan server. Di awal proses, fungsi ini melakukan pemeriksaan validasi apakah server sedang aktif atau mati; jika terdeteksi mati, sistem akan langsung mengalihkan pengguna ke mode pemulihan darurat (backup_mode). Apabila server aktif, fungsi akan meminta pengguna memasukkan nama pengguna (username) dan kata sandi utama (master password), lalu secara simultan mencocokkan keberadaan data akun tersebut pada basis data server dan berkas JSON lokal klien. Selanjutnya, program memanfaatkan metode KDF (PBKDF2)

untuk mengolah kata sandi masukan bersama komponen salt lokal guna menghasilkan kunci turunan yang digunakan untuk membuka dekripsi komponen pecahan rahasia lokal (local share).

Setelah pecahan rahasia lokal berhasil diekstraksi dan diverifikasi, fungsi ini melangkah ke tahap rekonstruksi kriptografi dengan mengambil komponen pecahan rahasia server dari basis data. Kedua pecahan tersebut kemudian digabungkan melalui perhitungan matematis interpolasi untuk menyusun kembali kunci induk utama (master key) asli. Kunci induk hasil rekonstruksi inilah yang langsung digunakan untuk mendekripsi gumpalan data brankas utama (vault_blob) yang dikirim oleh server agar dapat diubah kembali menjadi list data teks biasa. Begitu seluruh rantai autentikasi dan dekripsi selesai dengan sukses, fungsi ini membuka menu interaktif berbasis perulangan yang memberikan hak akses penuh bagi pengguna untuk mengelola kata sandi mereka secara langsung, mulai dari melihat, menambah, memperbarui, hingga menghapus data.

xii. Backup Mode

```
def backup_mode():
    username = input("Username: ")
    password = input("Master Password: ")
    recovery = input("Recovery Share: ")

    local = client.load_local(username)
    if not local:
        print("No local backup")
        return

    try:
        kdf_key = client.derive_key(password, local["salt"])

        local_share = json.loads(
            client.decrypt(local["enc_share"], kdf_key,
local["nonce"])
        )

        recovery_share = json.loads(recovery)

        master_key = client.reconstruct_secret(local_share,
recovery_share)

        vault = json.loads(
            client.decrypt(local["backup_vault"], master_key,
local["backup_nonce"])
        )

        view_passwords(vault)

    except:
        print("BACKUP FAILED")
```

Code 20. fungsi backup_mode di main.py

Fungsi backup_mode() bertindak sebagai mekanisme ketahanan sistem (fault tolerance) atau jalur darurat yang memungkinkan pengguna untuk tetap dapat mengakses brankas kata sandi mereka secara mandiri saat server sedang luring (offline). Pada awal eksekusi, fungsi ini meminta pengguna memasukkan tiga data penting, yaitu nama pengguna (username), kata sandi utama (master password), dan pecahan kode pemulihan (recovery share) yang telah disimpan pengguna sejak awal pendaftaran akun. Langkah pertama yang dilakukan sistem adalah memeriksa keberadaan berkas

penyimpanan JSON lokal klien di dalam komputer; jika file cadangan lokal tersebut tidak ditemukan atau terhapus, fungsi akan langsung menghentikan proses dan menampilkan pesan kesalahan.

Apabila berkas cadangan lokal tersedia, fungsi ini akan masuk ke dalam blok pengujian (try-except) untuk memproses rantai dekripsi dan rekonstruksi kunci secara mandiri tanpa melibatkan data server sama sekali. Program mula-mula menurunkan kunci dekripsi dari password yang diinput bersama salt lokal via KDF untuk membuka pecahan kunci rahasia lokal (local share). Selanjutnya, pecahan lokal tersebut digabungkan bersama pecahan pemulihan (recovery share) masukan pengguna ke dalam fungsi rekonstruksi Shamir untuk memulihkan kunci utama (master key) asli. Setelah kunci utama berhasil disusun kembali, program menggunakannya untuk mendekripsi data brankas cadangan terenkripsi (backup_vault) yang melekat di dalam JSON lokal. Jika seluruh proses pembukaan enkripsi ini berhasil dan valid, isi brankas kata sandi akan langsung ditampilkan ke layar pengguna untuk dibaca.

4. Pengujian Program

sebelum mulai pengujian, ada beberapa bagian kode yang perlu diubah yaitu:

fungsi create_vault

```
def create_vault():
    username = input("Username: ").strip()
    password = input("Master Password: ").strip()

    if not username or not password:
        print("Invalid input")
        return

    if server.get_vault(username):
        print("Username already exists. Use login instead.")
        return

    if not server.SERVER_ACTIVE:
        print("Server offline. Cannot create vault.")
        return

    master_key = get_random_bytes(16)

    local_share, server_share, recovery_share =
client.split_secret(master_key)

    salt = get_random_bytes(16)
    kdf_key = client.derive_key(password, salt)

    enc_local, nonce_local = client.encrypt(
        json.dumps(local_share).encode(),
        kdf_key
    )

    vault = []
    print("INITIAL VAULT:", vault)

    enc_vault, nonce_vault = client.encrypt(
        json.dumps(vault).encode(),
        master_key
    )
    print("ENCRYPTED VAULT:", enc_vault)
    print("NONCE:", nonce_vault)
    server.save_vault(
```

```

        username,
        json.dumps(server_share),
        enc_vault,
        nonce_vault
    )

    client.save_local(
        username,
        enc_local,
        nonce_local,
        salt,
        enc_vault,
        nonce_vault
    )

    print("\nVAULT CREATED SUCCESSFULLY")
    print("RECOVERY SHARE (SAVE THIS ONCE):")
    print(json.dumps(recovery_share))
    print("MASTER KEY (DEBUG):", master_key.hex())
    print("LOCAL:", local_share)
    print("SERVER:", server_share)
    print("RECOVERY:", recovery_share)

```

Code 21. fungsi create_vault di main.py untuk pengujian

```

def normal_mode():
    if not server.SERVER_ACTIVE:
        print("Server offline → switching to backup mode")
        backup_mode()

    username = input("Username: ")
    password = input("Master Password: ")

    srv = server.get_vault(username)
    if not srv:
        print("User not found")
        return

    local = client.load_local(username)
    if not local:
        print("Local data missing")
        return

    try:
        kdf_key = client.derive_key(password, local["salt"])

        local_share = json.loads(
            client.decrypt(local["enc_share"], kdf_key,
local["nonce"])
        )

        print("MASTER PASSWORD VALID")
        print("DECRYPTED LOCAL SHARE:", local_share)

    except:
        print("AUTH FAILED")
        print("DECRYPTED LOCAL SHARE:", local_share)
        return

    try:
        server_share = json.loads(srv["server_share"])
        print("[DEBUG] SERVER SHARE:", server_share)

```

```

        master_key = client.reconstruct_secret(local_share,
server_share)
        print("[DEBUG] MASTER KEY RECONSTRUCTED:", master_key.hex())

        vault = json.loads(
            client.decrypt(srv["vault_blob"], master_key,
srv["nonce"])
        )
        print("[DEBUG] DECRYPTED VAULT:", vault)
    except Exception as e:
        print("DECRYPT ERROR:", e)
        return

    while True:
        print("\n1. View")
        print("2. Add")
        print("3. Update")
        print("4. Delete")
        print("5. Exit")

        ch = input("> ")

        if ch == "1":
            view_passwords(vault)
        elif ch == "2":
            add_password(vault, master_key, username, server_share,
local)
        elif ch == "3":
            update_password(vault, master_key, username,
server_share, local)
        elif ch == "4":
            delete_password(vault, master_key, username,
server_share, local)
        elif ch == "5":
            break

```

Code 22. fungsi normal_mode di main.py untuk pengujian

```

        enc, nonce = client.encrypt(json.dumps(vault).encode(), key)
        server.save_vault(username, json.dumps(server_share), enc,
nonce)

        enc, nonce = client.encrypt(json.dumps(vault).encode(), key)
        print("\n[DEBUG] NEW ENCRYPTION AFTER ADD")
        print("[DEBUG] NONCE:", nonce.hex())
        print("[DEBUG] ENCRYPTED VAULT:", enc.hex()[:60], "...")
        server.save_vault(username, json.dumps(server_share), enc,
nonce)
        print("[DEBUG] VAULT SAVED TO SERVER (UPDATED VERSION)")

```

Code 23. perubahan fungsi add_password, update_password, delete_password di main.py untuk pengujian

```

def backup_mode():
    username = input("Username: ")
    password = input("Master Password: ")
    recovery = input("Recovery Share: ")

    local = client.load_local(username)
    if not local:
        print("No local backup")
        return

```

```

try:
    kdf_key = client.derive_key(password, local["salt"])

    local_share = json.loads(
        client.decrypt(local["enc_share"], kdf_key,
local["nonce"])
    )

    recovery_share = json.loads(recovery)
    print("[DEBUG] LOCAL SHARE DECRYPTED:", local_share)
    print("[DEBUG] RECOVERY SHARE USED:", recovery_share)
    master_key = client.reconstruct_secret(local_share,
recovery_share)
    print("[DEBUG] MASTER KEY RECONSTRUCTED:", master_key.hex())
    vault = json.loads(
        client.decrypt(local["backup_vault"], master_key,
local["backup_nonce"])
    )

    view_passwords(vault)

except:
    print("BACKUP FAILED")

```

Code 24. perubahan fungsi backup_mode di main.py untuk pengujian

```

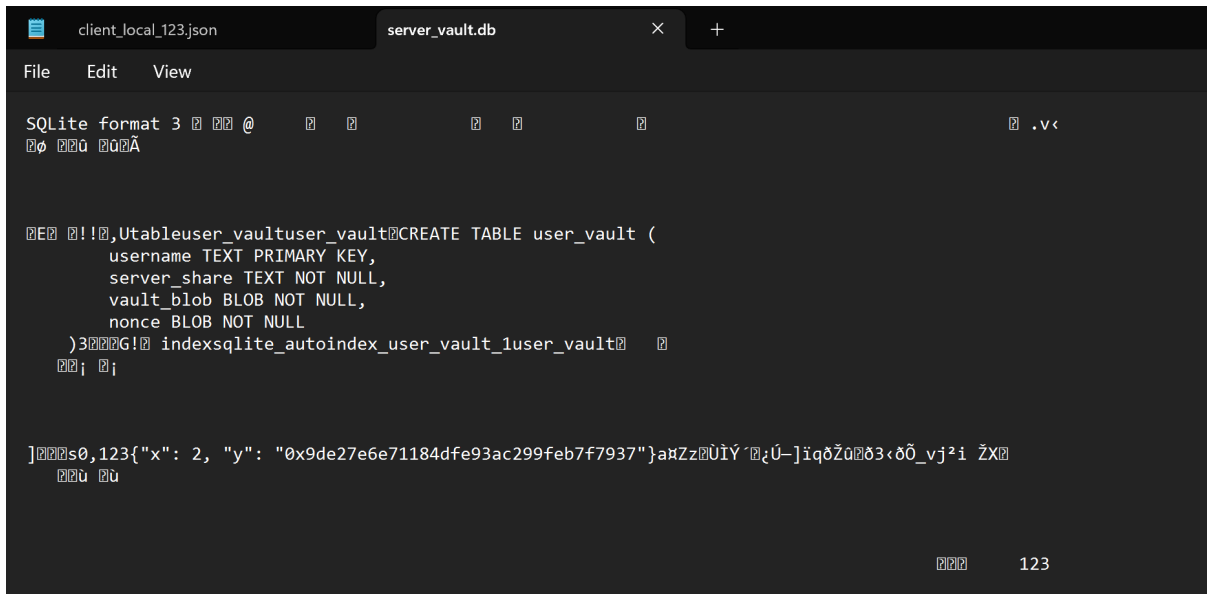
PS D:\Tugas4-II4021-2026> py main.py
1. Create Vault
2. Login
3. Backup Mode
4. Exit
5. Toggle Server
> 1
Username: 123
Master Password: 1234
INITIAL VAULT: []
ENCRYPTED VAULT: b'a\xa4Zz\x11\xd9\xcc\xdd\xb4\x04\xbf\xda\x97]\xefq\xf0\x8e'
NONCE: b'\xfb\xf0\xf03\x8b\xf0\xd5_vj\xb2i\x07\x8eX\x1d'

VAULT CREATED SUCCESSFULLY
RECOVERY SHARE (SAVE THIS ONCE):
{"x": 3, "y": "0x201caf5ce34b1eb0dc684d96fa07c519"}
MASTER KEY (DEBUG): 996e1c918cb2ac9a0233e1b1ce6ee140
LOCAL: {'x': 1, 'y': '0x1ba84d7ffee57d4c4af005a8dcf72d22'}
SERVER: {'x': 2, 'y': '0x9de27e6e71184dfe93ac299feb7f7937'}
RECOVERY: {'x': 3, 'y': '0x201caf5ce34b1eb0dc684d96fa07c519'}

1. Create Vault
2. Login
3. Backup Mode
4. Exit
5. Toggle Server
> |

```

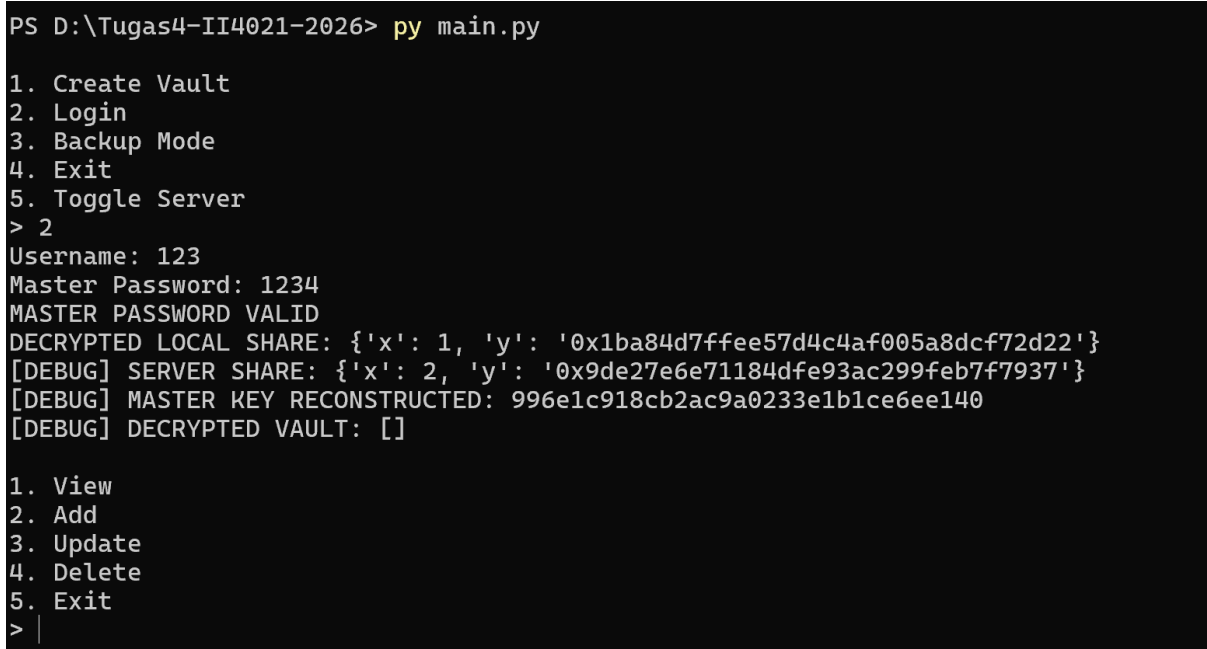
Gambar 1. bukti uji pembuatan vault 1



Gambar 2. isi server_vault.db setelah pembuatan vault



Gambar 3. isi client_local_123 setelah pembuatan vault



Gambar 4. dekripsi local share setelah input master password yang benar

```

1. Create Vault
2. Login
3. Backup Mode
4. Exit
5. Toggle Server
> 2
Username: 123
Master Password: 123
AUTH FAILED
Traceback (most recent call last):
  File "D:\Tugas4-II4021-2026\main.py", line 260, in normal_mode
    client.decrypt(local["enc_share"], kdf_key, local["nonce"])
  File "D:\Tugas4-II4021-2026\client.py", line 59, in decrypt
    return cipher.decrypt_and_verify(ct, tag)
           ^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\Jason Samuel\AppData\Local\Programs\Python\Python312\Lib\site-packages\Crypto\Cipher\_mode_gcm.py", line 567, in decrypt_and_verify
    self.verify(received_mac_tag)
  File "C:\Users\Jason Samuel\AppData\Local\Programs\Python\Python312\Lib\site-packages\Crypto\Cipher\_mode_gcm.py", line 508, in verify
    raise ValueError("MAC check failed")
ValueError: MAC check failed

During handling of the above exception, another exception occurred:

Traceback (most recent call last):
  File "D:\Tugas4-II4021-2026\main.py", line 376, in <module>
    main()
  File "D:\Tugas4-II4021-2026\main.py", line 363, in main
    normal_mode()
  File "D:\Tugas4-II4021-2026\main.py", line 268, in normal_mode
    print("DECRYPTED LOCAL SHARE:", local_share)
                                   ^^^^^^^^^^^^^
UnboundLocalError: cannot access local variable 'local_share' where it is not associated with a value
PS D:\Tugas4-II4021-2026> |

```

Gambar 5. dekripsi local share setelah input master password yang salah

```
PS D:\Tugas4-II4021-2026> py main.py

1. Create Vault
2. Login
3. Backup Mode
4. Exit
5. Toggle Server
> 2
Username: 123
Master Password: 1234
MASTER PASSWORD VALID
DECRYPTED LOCAL SHARE: {'x': 1, 'y': '0x1ba84d7ffee57d4c4af005a8dcf72d22'}
[DEBUG] SERVER SHARE: {'x': 2, 'y': '0x9de27e6e71184dfe93ac299feb7f7937'}
[DEBUG] MASTER KEY RECONSTRUCTED: 996e1c918cb2ac9a0233e1b1ce6ee140
[DEBUG] DECRYPTED VAULT: []

1. View
2. Add
3. Update
4. Delete
5. Exit
> |
```

Gambar 6. testing normal_mode

Gambar 7. testing normal mode dengan vault yang terisi

Gambar 8. testing normal mode dengan master password yang salah

```

1. View
2. Add
3. Update
4. Delete
5. Exit
> 2
Service: Github
Username: 123
1. Manual Password
2. Auto Generate
Choice: 1
Password: 12345
Catatan: Test 1

[DEBUG] NEW ENCRYPTION AFTER ADD
[DEBUG] NONCE: 251c2f546fdb5c5f2db4cca06399bb4d
[DEBUG] ENCRYPTED VAULT: c5e42b840e63daf9df77276796f0245077ecafa2cda481b4777414a6483c ...
[DEBUG] VAULT SAVED TO SERVER (UPDATED VERSION)
ADDED

1. View
2. Add
3. Update
4. Delete
5. Exit
> 1

--- YOUR VAULT ---
[0] Github | 123 | 12345 | Catatan: Test 1

1. View
2. Add
3. Update
4. Delete
5. Exit
> |

```

Gambar 9. penambahan password manual ke vault

```

1. View
2. Add
3. Update
4. Delete
5. Exit
> 2
Service: Github
Username: 123
1. Manual Password
2. Auto Generate
Choice: 2
Length: 12
Catatan: Test 2

[DEBUG] NEW ENCRYPTION AFTER ADD
[DEBUG] NONCE: 83d8a7e7cf59a4eaa2faecdbf9bb1d1e
[DEBUG] ENCRYPTED VAULT: 87ec5be929fbbb7f1c5d4755575d16ca58a878459da0a2b5d524455dfa84 ...
[DEBUG] VAULT SAVED TO SERVER (UPDATED VERSION)
ADDED

1. View
2. Add
3. Update
4. Delete
5. Exit
> |

```

Gambar 10. penambahan password auto generated ke vault

```

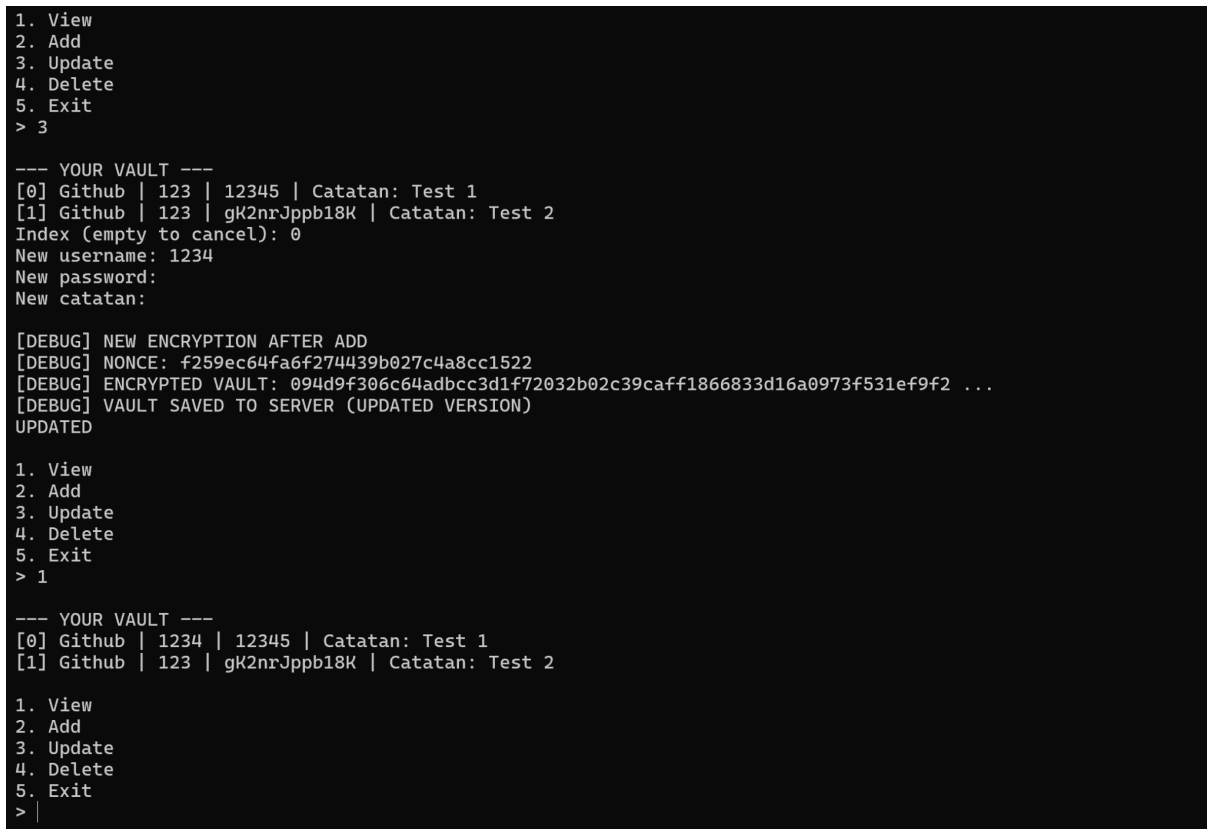
server_vault.db  client_local_123.json  x  +
File Edit View

{"enc_share": "aZ6A0sjIyg1arM5aKwkyVy3t2rXB0CeM12MbGLE6Htdw0Zk7Iiuee60h/hD/ArHeQvm1MwdgZMPrvkytBerbCrzZQ==", "nonce": "px108JZTjYOMCGGTm4x07g==", "salt":
"vHmYc6Ic101DeInNoogxwA==", "backup_vault":
"hi+xb6Sn7u38cXUdV10Wyl1oeEwdoKK11SRFXfqE+w0aq9CNagkTVXoZ1Lck/thiRcyjY6c7u2ttVGkisMoa1M7lhH2g5KG757psqBeDBcuVUfnIUX4MeipdafJ/GNCSKNTB5Xj/UnUBKzA90Lq69Hj8+
00uZ7rLLSw4QSANFw44tE1XKw2MhaHNUWmq8SQ8zfK2U/okO+5sjPv2KuaKucFuPY1d0ya/1pwJRMh6AREPb7ane1RTZGXTMTsB+L+ebHX13/ZDs3E", "backup_nonce": "g91n589Zp0q1+uzb+bsdHg=="}
```

Gambar 11. Isi local vault setelah penambahan password pada gambar 9&10



Gambar 12. Isi server vault setelah penambahan password pada gambar 9&10



Gambar 13. perubahan password

```

1. View
2. Add
3. Update
4. Delete
5. Exit
> 4

--- YOUR VAULT ---
[0] Github | 1234 | 12345 | Catatan: Test 1
[1] Github | 123 | gK2nrJppb18K | Catatan: Test 2
Index (empty to cancel): 0

[DEBUG] NEW ENCRYPTION AFTER ADD
[DEBUG] NONCE: 956369d3926e85a22628d298223f02a2
[DEBUG] ENCRYPTED VAULT: d716677dab813ba2925210949578bf08bee9dbaa9079affa723782b608bc ...
[DEBUG] VAULT SAVED TO SERVER (UPDATED VERSION)
DELETED

1. View
2. Add
3. Update
4. Delete
5. Exit
> 1

--- YOUR VAULT ---
[0] Github | 123 | gK2nrJppb18K | Catatan: Test 2

1. View
2. Add
3. Update
4. Delete
5. Exit
> |

```

Gambar 14. penghapusan password

```

PS D:\Tugas4-II4021-2026> py main.py

1. Create Vault
2. Login
3. Backup Mode
4. Exit
5. Toggle Server
> 3
Username: 123
Master Password: 1234
Recovery Share: {"x": 3, "y": "0x201caf5ce34b1eb0dc684d96fa07c519"}
[DEBUG] LOCAL SHARE DECRYPTED: {'x': 1, 'y': '0x1ba84d7ffee57d4c4af005a8dcf72d22'}
[DEBUG] RECOVERY SHARE USED: {'x': 3, 'y': '0x201caf5ce34b1eb0dc684d96fa07c519'}
[DEBUG] MASTER KEY RECONSTRUCTED: 996e1c918cb2ac9a0233e1b1ce6ee140

--- YOUR VAULT ---
[0] Github | 123 | gK2nrJppb18K | Catatan: Test 2

1. Create Vault
2. Login
3. Backup Mode
4. Exit
5. Toggle Server
> |

```

Gambar 15. Testing Backup Mode

```

1. Create Vault
2. Login
3. Backup Mode
4. Exit
5. Toggle Server
> 2
Server offline → switching to backup mode
Username: 123
Master Password: 1234
Recovery Share: {"x": 3, "y": "0x201caf5ce34b1eb0dc684d96fa07c519"}
[DEBUG] LOCAL SHARE DECRYPTED: {'x': 1, 'y': '0x1ba84d7ffee57d4c4af005a8dcf72d22'}
[DEBUG] RECOVERY SHARE USED: {'x': 3, 'y': '0x201caf5ce34b1eb0dc684d96fa07c519'}
[DEBUG] MASTER KEY RECONSTRUCTED: 996e1c918cb2ac9a0233e1b1ce6ee140

--- YOUR VAULT ---
[0] Github | 123 | gK2nrJppb18K | Catatan: Test 2
Username: 123
Master Password: 1234
User not found

1. Create Vault
2. Login
3. Backup Mode

```

Gambar 16. Toggle Server Off dan penggunaan Backup Mode di kondisi server off

```

> {"x": 3, "y": "0x201caf5ce34b1eb0dc684d96fa07c512"}

1. Create Vault
2. Login
3. Backup Mode
4. Exit
5. Toggle Server
> 3
Username: 123
Master Password: 1234
Recovery Share: {"x": 3, "y": "0x201caf5ce34b1eb0dc684d96fa07c512"}
[DEBUG] LOCAL SHARE DECRYPTED: {'x': 1, 'y': '0x1ba84d7ffee57d4c4af005a8dcf72d22'}
[DEBUG] RECOVERY SHARE USED: {'x': 3, 'y': '0x201caf5ce34b1eb0dc684d96fa07c512'}
[DEBUG] MASTER KEY RECONSTRUCTED: 196e1c918cb2ac9a0233e1b1ce6ee12a
BACKUP FAILED

1. Create Vault
2. Login
3. Backup Mode
4. Exit
5. Toggle Server
> |

```

Gambar 17. Penggunaan Backup Mode di dengan recovery share yang salah


```
LOCAL: {'x': 1, 'y': '0x1ba84d7ffee57d4c4af005a8dcf72d22'}  
SERVER: {'x': 2, 'y': '0x9de27e6e71184dfe93ac299feb7f7937'}  
RECOVERY: {'x': 3, 'y': '0x201caf5ce34b1eb0dc684d96fa07c519'}
```

menunjukkan local share, server share, dan recovery share.

- Perhatikan bahwa recovery share ditampilkan dalam format teks yang memuat koordinat share dan nilai share.

pada [gambar 1](#), bagian

```
VAULT CREATED SUCCESSFULLY  
RECOVERY SHARE (SAVE THIS ONCE):  
{ "x": 3, "y": "0x201caf5ce34b1eb0dc684d96fa07c519" }
```

ini menunjukkan bahwa recovery share ditampilkan dalam format teks yang memuat koordinat share dan nilai share.

- Perhatikan bahwa server hanya menyimpan server share, vault terenkripsi, nonce vault, dan metadata yang diperlukan.

pada [gambar 2](#) bagian

```
CREATE TABLE user_vault (  
  username TEXT PRIMARY KEY,  
  server_share TEXT NOT NULL,  
  vault_blob BLOB NOT NULL,  
  nonce BLOB NOT NULL  
)  
CREATE INDEX indexsqlite_autoindex_user_vault_1 ON user_vault (username)
```

terlihat bahwa server menyimpan metadata berupa username sebagai primary key, lalu ada juga server share, vault_blob yaitu vault terenkripsi, dan nonce yaitu nonce vault.

2. Uji penyimpanan dan perlindungan local share

- Perhatikan bahwa local share tidak disimpan dalam bentuk asli.
- Perhatikan bahwa local share berhasil dienkripsi menggunakan kunci turunan dari master password.

[gambar 3](#) menunjukkan bahwa local share tidak disimpan dalam bentuk asli, melainkan dalam bentuk data terenkripsi.

- Masukkan master password yang benar, perhatikan bahwa local share berhasil didekripsi.

hasilnya seperti yang ada di [gambar 4](#), di mana local share berhasil didekripsi.

- Masukkan master password yang salah, perhatikan bahwa local share gagal didekripsi atau akses ditolak.

Jika input master password yang salah, hasilnya akan seperti [gambar 5](#), di mana muncul AUTH FAILED dan terjadi program error karena variabel local_share tidak memiliki nilai(value).

3. Uji Akses Normal

- Buka vault dengan master password yang benar ketika server dapat diakses, perhatikan bahwa sistem menggunakan kombinasi local share dan server share.

terlihat pada [gambar 6](#) local share & server share digunakan untuk merekonstruksi master key

- Perhatikan bahwa master key berhasil direkonstruksi dari dua share valid.

terlihat pada [gambar 6](#) bahwa master key berhasil direkonstruksi

- Perhatikan bahwa vault berhasil didekripsi dan isi vault dapat ditampilkan.

terlihat pada [gambar 7](#) bahwa vault berhasil didekripsi dan isi vault dapat ditampilkan

- Masukkan master password yang salah atau gunakan share yang tidak valid, perhatikan bahwa dekripsi vault gagal dan data password tidak ditampilkan.

untuk testing ini ubah fungsi normal_mode bagian:

```
except:
    print("AUTH FAILED")
    print("DECRYPTED LOCAL SHARE:", local_share)
    return
```

menjadi:

```
except:
    print("AUTH FAILED")
    print("[DEBUG] DECRYPTED VAULT:", vault)
    return
```

terlihat pada [gambar 8](#) bahwa jika master password salah maka dekripsi vault gagal dan terjadi program error karena variabel vault tidak memiliki nilai (tidak dapat ditampilkan)

4. Uji Penambahan Data Password

- Tambahkan data password dengan input manual, perhatikan bahwa data berhasil masuk ke vault.

terlihat pada [gambar 9](#) penambahan password dengan input manual berhasil dan dapat ditampilkan

- Tambahkan data password dengan pembangkitan otomatis menggunakan CSPRNG, perhatikan bahwa password berhasil dibangkitkan sesuai panjang yang diminta.

terlihat pada [gambar 10](#) penambahan password auto generated berhasil dan password memiliki panjang 12 huruf seperti yang diminta.

- Perhatikan bahwa setelah penambahan data, vault dienkripsi ulang

menggunakan nonce baru dan disimpan kembali ke server.
terlihat pada [gambar 9](#) & [gambar 10](#) setelah penambahan ke vault noncenya berbeda, vault kembali terenkripsi.
isi server vault setelah perubahan pada [gambar 9](#) & [gambar 10](#) berubah dari:

```
]P0P0s0,123{"x": 2, "y": "0x9de27e6e71184dfe93ac299feb7f7937"}aHZZ0ÙÏÝ`BzÚ-JiqðŽú0ð3<ðð_vj²i ŽX0P
```

menjadi : ([gambar 12](#))

```
,0000s0,123{"x": 2, "y": "0x9de27e6e71184dfe93ac299feb7f7937"}t1[e]0m00]G0W]0EX'xEP qμð$E]0,00s<ð0j 0Uz00*0p0beIec$;=kmT1"0É00Ia,} a1=K01"0f0E•Q0ÉQ~z*]10000' (0AaxYRu0+0=8**0xu0J.I0E-,8Y004\8000\~620+5E*~A0o7E00e0i+3i000*c0,05wL0p*p00!00D=NU00e'M'-LÀ1 0py±×(000IÄf0$çIY0e00i0u=000
```

- Perhatikan bahwa backup vault lokal juga diperbarui setelah perubahan pada mode normal.

saat vault kosong data di vault lokal adalah:

```
{"enc_share": "a26A0sjIyglarM5akWkyVy3t2rX80CeM12MbGLeL6Htdw0Zk7Iiue60h/hD/ArHeQvmI0wdgZMPrvkytBerbCrzZQ==", "nonce": "px108JZTjY0MCGGT4x07g==", "salt": "vHmYc6ic10iDeInNoogxWA==", "backup_vault": "YaRaehHZn20BL/a113vcfCO", "backup_nonce": "+w/wM4vw1V92arJp845YHQ=="}
```

setelah penambahan di [gambar 9](#) & [gambar 10](#) isinya berubah menjadi: ([gambar 11](#))

```
{"enc_share": "a26A0sjIyglarM5akWkyVy3t2rX80CeM12MbGLeL6Htdw0Zk7Iiue60h/hD/ArHeQvmI0wdgZMPrvkytBerbCrzZQ==", "nonce": "px108JZTjY0MCGGT4x07g==", "salt": "vHmYc6ic10iDeInNoogxWA==", "backup_vault": "h+xb6Sn7u38cXudV10wy11oeEwdokK11SRFXfqE+w0aq9CNagkTVXoZ1Lck/th1RcyjY6c7u2tVGk1sMoa1M71hh2g5KG757psqBe0BcuVUfnIUX4Me1pdaFJ/GNCSKNTB5Xj/UnUBKzA90Lq69Hj8+0ouz7rLLSw4WQ5ANFw44E1XK02Mhal0Ukyq8SQ8zfk2U/ok0+5s3Pv2KUaKucFuPY1d0ya/1pwJRMh6AREP07ane1RTZGXTMTsB+L+eb0X13/ZDs3E", "backup_nonce": "g91n589Zp0q1+uzb+bsdlfg=="}
```

membuktikan bahwa backup vault lokal juga diperbarui.

5. Uji Pengubahan dan Penghapusan Data Password

- Ubah salah satu data password yang tersimpan, perhatikan bahwa perubahan berhasil tersimpan setelah vault dienkripsi ulang.

terlihat pada [gambar 13](#) ketika di view setelah perubahan, yaitu setelah enkripsi ulang, perubahan berhasil tersimpan

- Hapus salah satu data password yang tersimpan, perhatikan bahwa data tersebut tidak lagi muncul setelah vault dibuka kembali.

terlihat pada [gambar 14](#) setelah password dihapus, ketika vault dibuka kembali password tersebut tidak lagi muncul.

- Perhatikan bahwa pembaruan hanya dilakukan pada mode normal.

terlihat pada [gambar 15](#) bahwa pada backup mode hanya memunculkan isi vault. tidak ada operasi untuk mengubah isi vault.

6. Uji Penyimpanan Vault di Server

- Perhatikan bahwa vault terenkripsi disimpan di SQLite sebagai BLOB.

terlihat pada [gambar 12](#) bahwa vault terenkripsi di vault di server (SQLite) tersimpan sebagai BLOB

- Perhatikan bahwa server tidak menyimpan nama layanan, username, password, atau catatan sebagai baris plaintext terpisah.

terlihat pada [gambar 12](#) bahwa di server nama layanan, username, password, dan catatan terenkripsi dalam satu plaintext, tidak terpisah.

- Perlihatkan bahwa server tidak menerima master key, local share, recovery share, maupun isi vault dalam bentuk plaintext.

terlihat pada [gambar 12](#) bahwa di server yang ada hanyalah:

username TEXT PRIMARY KEY,
server_share TEXT NOT NULL,
vault_blob BLOB NOT NULL,
nonce BLOB NOT NULL

dan di plaintext pun hanya ada username, server share, dan data vault blob dan nonce yang terenkripsi.

7. Uji Mode Backup

- Simulasikan kondisi server tidak dapat diakses.

Berhasil, di [gambar 16](#) terlihat bahwa server berhasil dimatikan dan saat mengakses normal mode yang memerlukan server aktif, program langsung pindah ke backup mode.

- Buka vault menggunakan master password dan recovery share, perlihatkan bahwa sistem menggunakan kombinasi local share dan recovery share.

Terlihat bahwa di [gambar 16](#) backup mode berhasil membuka vault, dan local share & recovery share bisa dipanggil, artinya keduanya digunakan untuk proses ini.

- Perlihatkan bahwa sistem menggunakan backup vault lokal terenkripsi sebagai sumber vault.

di kode program fungsi backup mode:

```
vault = json.loads(  
    client.decrypt(local["backup_vault"], master_key, local["backup_nonce"])  
)  
view_passwords(vault)
```

artinya dilakukan dekripsi untuk backup vault, lalu hasil dekripsi adalah yang ditampilkan, seperti apa yang terjadi pada [gambar 16](#), dimana view_passwords berhasil menampilkan isi backup vault.

- Perlihatkan bahwa data password dapat dilihat pada mode backup.

terlihat pada [gambar 16](#) bahwa backup mode berhasil menampilkan isi vault

- Perlihatkan bahwa pengguna tidak dapat menambah, mengubah, atau menghapus data password pada mode backup.

terjadi pada [gambar 16](#) bahwa backup mode hanya menampilkan isi vault, tetapi tidak ada operasi untuk melakukan perubahan pada data seperti penambahan, update, dan penghapusan. program meminta input username dan password lagi, ini karena program melanjutkan pemanggilan normal mode, tapi karena server tidak aktif maka tidak bisa

8. Uji Kegagalan Pemulihan

- Masukkan recovery share yang salah, perlihatkan bahwa rekonstruksi master key atau dekripsi vault gagal.

terlihat di [gambar 17](#) bahwa jika menggunakan recovery share yang salah maka backup akan gagal

- Gunakan backup vault yang tidak sesuai atau sudah dimodifikasi, perhatikan bahwa dekripsi AES-128-GCM gagal karena validasi autentikasi tidak sesuai.

mengubah backup vault

Dari
"1xZnfauB06K9
Menjadi
"2ZnfauB06K9

terlihat seperti [gambar 18](#) bahwa program error karena gagal validasi

- Perhatikan bahwa sistem tidak menampilkan data password ketika proses dekripsi gagal.

terlihat di [gambar 18](#) bahwa ketika proses dekripsi gagal maka terjadi program error

5. Kesimpulan dari Hasil Implementasi

Program ini berhasil mengimplementasikan password manager dengan shamir secret sharing (2,3). Program ini memiliki fitur utama:

- Shamir's Secret Sharing (2,3): Memecah kunci utama (master key) menjadi 3 bagian, yaitu untuk lokal, server, dan pemulihan, di mana diperlukan minimal 2 bagian untuk merekonstruksi kunci.
- Enkripsi (AES-GCM & PBKDF2): Mengamankan data brankas dengan enkripsi AES-GCM dan mengamankan kunci lokal menggunakan kata sandi utama yang diperkuat dengan metode PBKDF2
- Manajemen Brankas (CRUD): Memungkinkan pengguna untuk membuat brankas baru (Create), serta melihat (View), menambah (Add), mengubah (Update), dan menghapus (Delete) data akun atau kata sandi di dalamnya.
- Backup Mode: Memungkinkan pengguna untuk tetap mengakses data brankas secara lokal menggunakan kunci pemulihan (recovery share) jika server sedang offline
- Pembuatan Kata Sandi Otomatis: fitur untuk membuat kata sandi acak secara otomatis saat menambahkan data baru dengan panjang yang dapat ditentukan.
- Penyimpanan Ganda: Menyimpan data brankas terenkripsi baik di sisi klien (local JSON) maupun di sisi server (SQLite database).

Program ini juga berhasil melalui testing dengan hasil yang baik. dimana program ini lulus semua testing yang dilakukan.

6. Daftar Pustaka

1. A. Shamir, "How to share a secret," Communications of the ACM, vol. 22, no. 11, pp. 612–613, Nov. 1979.
2. D. A. McGrew and J. Viega, "The Galois/Counter Mode of Operation (GCM)," Submission to NIST Modes of Operation Process, pp. 1–44, May 2004.
3. M. Dworkin, "Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC," National Institute of Standards and Technology (NIST), Gaithersburg, MD, USA, NIST Special Publication 800-38D, Nov. 2007.
4. B. Kaliski, "PKCS #5: Password-Based Cryptography Specification Version 2.0," RFC 2898, Sep. 2000.
5. K. Moriarty, B. Kaliski, and A. Rusch, "PKCS #5: Password-Based Cryptography Specification Version 2.1," RFC 8018, Internet Engineering Task Force (IETF), 2017.
6. E. Barker and J. Kelsey, "Recommendation for Random Number Generation Using Deterministic Random Bit Generators," National Institute of Standards and Technology (NIST), Gaithersburg, MD, USA, NIST Special Publication 800-90A Rev. 1, Jun. 2015.

7. Lampiran

1. Pranala Repositori

Berikut adalah link repository release pada github:

<https://github.com/JSamuelN/Tugas4-II4021-2026>

2. Pranala Video Demo

Berikut adalah link video demo:

<https://youtu.be/R7nTe0eMaR8>

3. Pembagian Tugas

Nama	NIM	Pembagian dan Pengerjaan Tugas
Jason Samuel	18223091	1. Membuat Laporan 2. Membuat Kode 3. Membuat Video Demo